

## แบบฝึกหัดกรณีศึกษา กรณีศึกษาที่ 2: ระบบตรวจสอบคะแนนซ้ำ

### คำชี้แจง

ให้นักศึกษาเลือกวิธีแก้ปัญหาที่เหมาะสมสำหรับแต่ละกรณีศึกษา จากนั้นดำเนินการดังต่อไปนี้

1. ระบุข้อมูลนำเข้าและผลลัพธ์ที่ต้องการ
2. กำหนดความหมายของขนาดข้อมูล (n)
3. เขียนแนวคิดหรือ Pseudocode
4. เขียนโปรแกรมภาษา Java
5. ระบุคำสั่งหลัก หรือ Basic Operation ที่ใช้วิเคราะห์
6. วิเคราะห์จำนวนครั้งที่คำสั่งหลักทำงาน
7. วิเคราะห์กรณี Best Case, Worst Case และ Average Case หากสามารถวิเคราะห์ได้
8. ระบุ Time Complexity ในรูป Big O, Omega, Theta
9. วิเคราะห์ Auxiliary Space Complexity
10. เสนอแนวทางปรับปรุงอัลกอริทึมให้มีประสิทธิภาพมากขึ้น

### กรณีศึกษาที่ 2: ระบบตรวจสอบคะแนนซ้ำ

อาจารย์มีรายการคะแนนสอบของนักศึกษา และต้องการตรวจสอบว่ามีคะแนนใดซ้ำกันหรือไม่ ตัวอย่างข้อมูล

```
int[] scores = {78, 85, 62, 90, 78, 74};
```

#### งานที่มอบหมาย

ให้นักศึกษาออกแบบ 2 วิธี

วิธีที่ 1

ใช้ลูปซ้อนกันเพื่อเปรียบเทียบคะแนนทุกคู่

วิธีที่ 2

ใช้ HashSet<Integer> เพื่อตรวจสอบคะแนนที่เคยพบแล้ว

เขียนเมธอด

```
static boolean hasDuplicateBruteForce(int[] scores)
```

และ

```
static boolean hasDuplicateHashSet(int[] scores)
```

### ส่วนที่ 1: โจทย์กรณีศึกษา

1. โจทย์กรณีศึกษา กรณีศึกษาที่ 2: ระบบตรวจสอบคะแนนซ้ำ

อาจารย์มีรายการคะแนนสอบของนักศึกษา และต้องการตรวจสอบว่ามีคะแนนใดซ้ำกันหรือไม่

ตัวอย่างข้อมูล: `int[] scores = {78, 85, 62, 90, 78, 74};`

## 2. ซอร์สโค้ดภาษา Java (Java Source Code)

ตอบ:

### 2.1. BruteForce

```
public class hasDuplicateBruteForce {  
  
    // เมธอดวิธีที่ 1: Brute Force  
  
    public static boolean hasDuplicateBruteForce(int[] scores) {  
  
        int n = scores.length;  
  
        for (int i = 0; i < n - 1; i++) {  
  
            for (int j = i + 1; j < n; j++) {  
  
                if (scores[i] == scores[j]) {  
  
                    return true; // พบคะแนนซ้ำ  
  
                }  
  
            }  
  
        }  
  
        return false; // ไม่พบคะแนนซ้ำ  
  
    }  
  
    public static void main(String[] args) {  
  
        int[] scores = {78, 85, 62, 90, 78, 74};  
  
        System.out.println("Result: " + hasDuplicateBruteForce(scores));  
  
    }  
  
}
```

### 2.2. HashSet

```
import java.util.HashSet;  
  
public class hasDuplicateHashSet {
```

```
// เมธอดวิธีที่ 2: HashSet

public static boolean hasDuplicateHashSet(int[] scores) {
    HashSet<Integer> seen = new HashSet<>();
    for (int score : scores) {
        if (seen.contains(score)) {
            return true; // พบคะแนนซ้ำ
        }
        seen.add(score);
    }
    return false; // ไม่พบคะแนนซ้ำ
}

public static void main(String[] args) {
    int[] scores = {78, 85, 62, 90, 78, 74};
    System.out.println("Result: " + hasDuplicateHashSet(scores));
}
}
```

### 3. สรุปคำถามวิเคราะห์ห้ท้ายกรณีศึกษา

#### คำถามวิเคราะห์

##### 1. กำหนดค่า (n)

ตอบ:

n คือ จำนวนข้อมูลคะแนนทั้งหมดในอาร์เรย์ (scores.length)

##### 2. วิเคราะห์ Best Case และ Worst Case ของแต่ละวิธี

ตอบ:

- วิธีที่ 1: Brute Force (ใช้ลูปซ้อน)

Best Case: พบคะแนนซ้ำกันทันทีในคู่แรกสุด (ตำแหน่ง Index 0 และ 1)

Worst Case: คะแนนทุกค่าไม่ซ้ำกันเลย หรือไปพบคะแนนซ้ำที่คู่สุดท้าย  
ทำให้ต้องเปรียบเทียบครบทุกคู่เป็นจำนวน  $\frac{n(n-1)}{2}$  ครั้ง

- วิธีที่ 2: HashSet

Best Case: พบคะแนนซ้ำตั้งแต่ตัวแรกๆ ทำให้หลุดออกจากลูปทันที

Worst Case: คะแนนทุกค่าไม่ซ้ำกันเลย ต้องวนลูปตรวจสอบและเพิ่มข้อมูลลงใน HashSet  
จนครบทุกตัวจำนวน  $n$  ครั้ง

### 3. ระบุ Time Complexity

ตอบ:

วิธีที่ 1 (Brute Force):

- Big O:  $O(n^2)$
- Big Omega:  $\Omega(1)$
- Big Theta :  $\Theta(n^2)$  (ในกรณี Worst / Average Case)

- วิธีที่ 2 (HashSet):

- Big O:  $O(n)$
- Big Omega:  $\Omega(1)$
- Big Theta:  $\Theta(n)$  (วิเคราะห์เจาะจงกรณี Worst Case)

### 4. ระบุ Auxiliary Space Complexity

ตอบ:

- วิธีที่ 1 (Brute Force):  $O(1)$   
(ใช้หน่วยความจำคงที่ ไม่มีการสร้างโครงสร้างข้อมูลเพิ่มตามจำนวน  $n$ )
- วิธีที่ 2 (HashSet):  $O(n)$   
(ต้องจองพื้นที่หน่วยความจำเพิ่มสำหรับเก็บข้อมูลลงใน HashSet สูงสุด  $n$  ตัว)

### 5. เปรียบเทียบข้อดีและข้อจำกัดของทั้งสองวิธี

ตอบ:

| หัวข้อเปรียบเทียบ | วิธีที่ 1: Brute Force                                                       | วิธีที่ 2: HashSet                                           |
|-------------------|------------------------------------------------------------------------------|--------------------------------------------------------------|
| ข้อดี             | ประหยัดหน่วยความจำ ( $O(1)$ Space) และเขียนโค้ดง่าย ไม่ต้องดึง Library เพิ่ม | ประมวลผลได้รวดเร็วมาก ( $O(n)$ Time)<br>รองรับข้อมูลขนาดใหญ่ |
| ข้อจำกัด          | ทำงานช้ามากเมื่อข้อมูลมีขนาดใหญ่ ( $O(n^2)$ Time)                            | สิ้นเปลืองหน่วยความจำเพิ่มขึ้น ( $O(n)$ Space)               |

6. อธิบาย Time–Space Trade-off ที่เกิดขึ้น

ตอบ: Time–Space Trade-off คือการ "ยอมแลกพื้นที่หน่วยความจำที่เพิ่มขึ้น เพื่อให้ได้เวลาในการประมวลผลที่เร็วขึ้น"

- วิธี Brute Force ประหยัดพื้นที่ ( $O(1)$  Space) แต่เสียเวลาทำงานนาน
- ในขณะที่วิธี HashSet ยอมเสียพื้นที่เก็บข้อมูลเพิ่ม ( $O(n)$  Space) แต่ทำให้ประมวลผลเสร็จเร็วขึ้นอย่างมาก ( $O(n)$  Time)

7. หากข้อมูลมี 1,000,000 ค่า ควรเลือกวิธีใด เพราะเหตุใด

ตอบ: ควรเลือกวิธีที่ 2 (HashSet)

เหตุผล:

- หากใช้ **Brute Force** เวลาในการทำงานกรณีแย่ที่สุดจะสูงถึงประมาณรอบ ซึ่งคอมพิวเตอร์ต้องใช้เวลาประมวลผลนานหลายนาทีหรือหลายชั่วโมง
- หากใช้ HashSet จะวนลูปประมวลผลเพียงประมาณ 1,000,000 รอบ ซึ่งคอมพิวเตอร์ทั่วไปสามารถทำงานเสร็จได้ในเวลาเพียงไม่กี่มิลลิวินาที

ส่วนที่ 2

1. ระบุข้อมูลนำเข้าและผลลัพธ์ที่ต้องการ

ตอบ: ข้อมูลนำเข้า (Input): อาร์เรย์ของจำนวนเต็ม `int[] scores {78, 85, 62, 90, 78, 74}`

ผลลัพธ์ที่ต้องการ (Output): ค่าตรรกศาสตร์ (Boolean)

true หากพบคะแนนสอบซ้ำกันอย่างน้อย 1 คู่

false หากคะแนนสอบทุกค่าไม่ซ้ำกันเลย

2. กำหนดความหมายของขนาดข้อมูล (n)

ตอบ: n คือ จำนวนข้อมูลคะแนนสอบทั้งหมดในอาร์เรย์ (scores.length)

3. เขียนแนวคิดหรือ Pseudocode

ตอบ:

วิธีที่ 1 (Brute Force) :แนวคิด: ใช้ลูป 2 ชั้นเพื่อจับคู่เปรียบเทียบคะแนนทุกคู่ โดยลูปนอก (i) วนตั้งแต่ตัวแรกถึงตัวก่อนสุดท้าย และลูปใน (j) วนตั้งแต่ตัวถัดไป (i+1) จนถึงตัวสุดท้าย เพื่อเช็คว่ามีคะแนนคู่ใดเท่ากันหรือไม่

Pseudocode:

```
Algorithm hasDuplicateBruteForce(scores):
```

```
    n = scores.length
```

```
    For i from 0 to n - 2 Do
```

```
        For j from i + 1 to n - 1 Do
```

```
            If scores[i] == scores[j] Then
```

```
                Return true
```

```
            End If
```

```
        End For
```

```
    End For
```

```
    Return false
```

วิธีที่ 2 (HashSet): แนวคิด: วนลูปอ่านคะแนนทีละตัว แล้วนำไปตรวจสอบใน HashSet (โครงสร้างข้อมูลที่เก็บค่าไม่ซ้ำ) หากพบคะแนนนั้นใน HashSet อยู่แล้ว แสดงว่าเคยพบมาแล้ว (ซ้ำ) ให้คืนค่า true ทันที แต่หากยังไม่เคยพบ ให้บันทึกคะแนนนั้นลงใน HashSet แล้วทำต่อจนจบ

Pseudocode:

Algorithm hasDuplicateHashSet(scores):

Create empty HashSet 'seen'

For each score in scores Do

    If seen.contains(score) Then

        Return true

    End If

    seen.add(score)

End For

Return false

#### 4. เขียนโปรแกรมภาษา Java

ตอบ:

Brute Force

```
public class hasDuplicateBruteForce {  
  
    // เมธอดวิธีที่ 1: Brute Force  
    public static boolean hasDuplicateBruteForce(int[] scores) {  
        int n = scores.length;  
        for (int i = 0; i < n - 1; i++) {  
            for (int j = i + 1; j < n; j++) {  
                if (scores[i] == scores[j]) {  
                    return true; // พบคะแนนซ้ำ  
                }  
            }  
        }  
        return false; // ไม่พบคะแนนซ้ำ  
    }  
  
    public static void main(String[] args) {  
        int[] scores = {78, 85, 62, 90, 78, 74};  
        System.out.println("Result: " + hasDuplicateBruteForce(scores));  
    }  
}
```

```
}
```

HashSet

```
import java.util.HashSet;

public class hasDuplicateHashSet {

    // เมธอดวิธีที่ 2: HashSet
    public static boolean hasDuplicateHashSet(int[] scores) {
        HashSet<Integer> seen = new HashSet<>();
        for (int score : scores) {
            if (seen.contains(score)) {
                return true; // พบคะแนนซ้ำ
            }
            seen.add(score);
        }
        return false; // ไม่พบคะแนนซ้ำ
    }

    public static void main(String[] args) {
        int[] scores = {78, 85, 62, 90, 78, 74};
        System.out.println("Result: " + hasDuplicateHashSet(scores));
    }
}
```

#### 5. ระบุคำสั่งหลัก หรือ Basic Operation ที่ใช้วิเคราะห์

**ตอบ:** วิธีที่ 1 (Brute Force): คำสั่งเปรียบเทียบค่าความเท่ากัน `scores[i] == scores[j]`  
วิธีที่ 2 (HashSet): คำสั่งค้นหาและเพิ่มข้อมูลใน HashSet คือ `seen.contains(score)`  
และ `seen.add(score)`

#### 6. วิเคราะห์จำนวนครั้งที่คำสั่งหลักทำงาน

**ตอบ:** วิธีที่ 1 (Brute Force):

- *WorstCase:*  $C_{worst}(n) = \frac{n(n-1)}{2} = \frac{n^2-n}{2}$  ครั้ง
- *BestCase:*  $C_{best}(n) = 1$  ครั้ง

วิธีที่ 2 (HashSet):

- *WorstCase:*  $C_{worst}(n) = n$  ครั้ง
- *BestCase:*  $C_{best}(n) = 1$  ครั้ง



7. วิเคราะห์กรณี Best Case, Worst Case และ Average Case หากสามารถวิเคราะห์ได้
- ตอบ:** วิธีที่ 1 (Brute Force): Best Case ทำงาน 1 ครั้ง | Worst Case ทำงาน  $\frac{n(n-1)}{2}$  ครั้ง
- Average Case ทำงานประมาณ  $\frac{n^2}{4}$  ครั้ง
- วิธีที่ 2 (HashSet): Best Case ทำงาน 1 ครั้ง | Worst Case ทำงาน n ครั้ง Average Case ทำงานประมาณ  $\frac{n}{2}$  ครั้ง
8. ระบุ Time Complexity ในรูป Big O, Omega, Theta
- ตอบ:** วิธีที่ 1 (Brute Force): Big O =  $O(n^2)$  | Big Omega =  $\Omega(1)$  | Big Theta =  $\Theta(n^2)$
- วิธีที่ 2 (HashSet): Big O =  $O(n)$  | Big Omega =  $\Omega(1)$  | Big Theta =  $\Theta(n)$
9. วิเคราะห์ Auxiliary Space Complexity
- ตอบ:** วิธีที่ 1 (Brute Force):  $O(1)$  (ใช้หน่วยความจำคงที่)
- วิธีที่ 2 (HashSet):  $O(n)$  (ต้องจองหน่วยความจำ RAM เพิ่มเพื่อสร้าง HashSet เก็บข้อมูลได้สูงสุด n ตัว)
10. เสนอแนวทางปรับปรุงอัลกอริทึมให้มีประสิทธิภาพมากขึ้น
- ตอบ:** ใช้เทคนิค Sorting (การเรียงลำดับข้อมูลก่อน)
- นำอาร์เรย์คะแนนมาเรียงลำดับจากน้อยไปมาก ใช้เวลา  $O(n \log n)$
  - วนลูป 1 รอบ ( $n - 1$  ครั้ง) เพื่อเปรียบเทียบคะแนนคู่ที่อยู่ติดกัน  $scores[i] == scores[i+1]$  ใช้เวลา  $O(n)$

**กลุ่มที่ 8 :** กรณีศึกษาที่ 2: ระบบตรวจสอบคะแนนซ้ำ

**สมาชิกในกลุ่ม**

กิตติมาภรณ์ รถทอง 68122250018

ภูวเดช ตรังคนสิน 6812225002

ชนาลักษณ์ จุลแดง 68122250025